

Día 1 – Parte 1: Negocio, revisión de archivos e identificación de problemas

DataCommerce GT

Integrantes:

- Daniella Marissa Navarro Araniva
- Diego Alejandro Escobar Barahona

Repositorio:

https://github.com/bapatata688/proyecto_final.git

1. Comprender el negocio

DataCommerce GT es una empresa que vende productos tecnológicos a través de 4 canales: tiendas físicas, comercio electrónico, ventas corporativas y WhatsApp.

Con el crecimiento de la empresa, la información quedó dispersa en distintos sistemas lo que provoca que la gerencia no tenga reportes confiables ni oportunos, y que cada departamento genere resultados que no coinciden entre sí.

¿Cómo se le dará solución? Construyendo una plataforma de datos que integre todas las fuentes en un único Data Warehouse, de manera que exista una sola fuente de verdad para generar los indicadores que la gerencia necesita para tomar decisiones.

2. Revisar archivos

En esa revisión se hará una exploración inicial de cada fuente: cuántas filas y columnas tiene, qué tipos de datos contiene, y una primera mirada a posibles inconsistencias (duplicados, nulos, formatos de fecha, textos mal escritos, etc.).

3. Identificar problemas

El objetivo de hacerlo así es que, para cuando avancemos en el Día 2 con la limpieza y estandarización, ya tengamos claro:

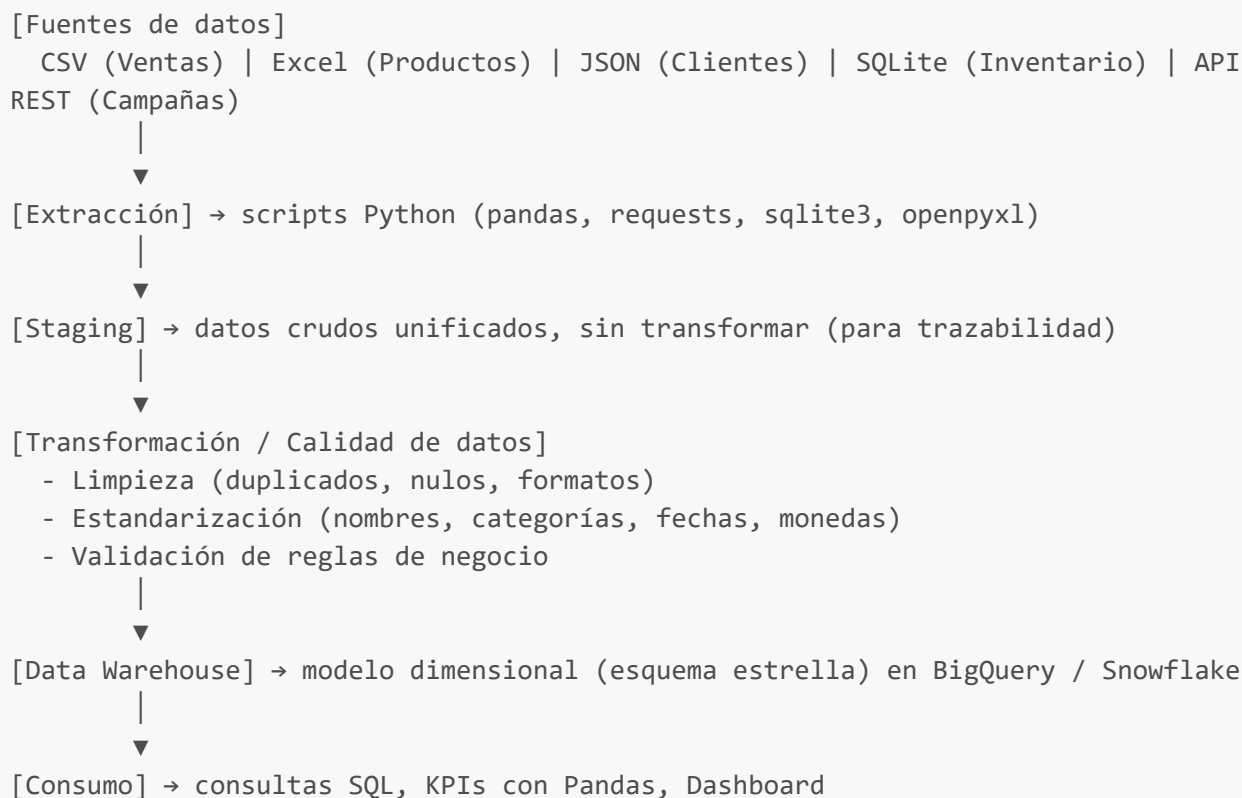
- Qué columnas tienen valores nulos o faltantes.
- Qué registros están duplicados.
- Qué formatos son inconsistentes entre fuentes (fechas, nombres, categorías, IDs).
- Qué reglas de limpieza y estandarización se deben aplicar y por qué.

De esta manera, la limpieza del Día 2 no se hace hara a ciegas, sino con base en los problemas ya identificados y documentados.

Día 1 – Parte 2: Arquitectura, modelo relacional y plan del ETL

DataCommerce GT

4. Diseño de la arquitectura de la solución propuesta



Stack tecnológico:

- Python + Pandas → extracción y transformación (ETL)
- BigQuery / Snowflake / SQLite → Data Warehouse
- SQL → consultas analíticas
- Herramienta de visualización (a definir en base a la información proporcionada) dashboard

Por qué esta arquitectura:

- Se separa staging de transformación para no perder los datos crudos originales.
- El pipeline se diseña para ser reproducible por que debe poder ejecutarse de nuevo sin intervención manual.
- El modelo dimensional facilita las consultas analíticas y el cálculo de KPIs para gerencia.

5. Modelo relacional (entidad-relación, preliminar)

A partir de lo que describe el enunciado del proyecto se tiene

Entidades identificadas:

- **Cliente**
- **Producto**
- **Venta**
- **Inventario**
- **Tienda/Canal**
- **Campaña**

Relaciones esperadas:

- Un Cliente puede tener muchas Ventas (1:N)
 - Un Producto puede aparecer en muchas Ventas (1:N)
 - Un Producto tiene inventario en varias Tiendas (N:M, vía Inventario)
 - Una Campaña puede influir en varias Ventas
-

6. Plan del ETL

Extracción (por fuente):

- CSV → `pandas.read_csv()`
- Excel → `pandas.read_excel()`
- JSON → `pandas.read_json()` o `json.load()` si hay anidamiento
- SQLite → `sqlite3` / `pandas.read_sql()`
- API REST → `requests.get()` con manejo de paginación/errores

Transformaciones esperadas a realizar, aunque variara dependiendo de los datos:

- Eliminar duplicados por clave primaria
- Normalizar fechas a formato `YYYY-MM-DD`
- Estandarizar nombres y categorías (mayúsculas, sin espacios extra)
- Definir regla para valores nulos (eliminar, imputar o marcar)
- Unificar IDs entre fuentes

Carga:

- Primero a staging (datos crudos)
- Luego, transformación y carga al modelo dimensional en el Data Warehouse